Tutorial 6 – Software Design Patterns

1. What are design patterns?

2. Why design patterns are seen to benefit the software development community?

3. What are creational patterns? What are they used for?

Creational patterns are a category of design patterns that deal with **object creation mechanisms**, aiming to make the process more flexible and dynamic. Instead of instantiating objects directly using the `new` keyword, creational patterns provide ways to create objects **that are decoupled from the specific classes being instantiated**.

They are used to:

- **Encapsulate object creation logic**.
- **Reduce system dependency** on concrete classes.
- **Promote flexibility and scalability** in how objects are created and managed.
- Support object reuse and resource optimization.

Common Types of Creational Patterns:

Pattern	Purpose	Example Use Case
Singleton	Ensures a class has **only one instance**, and provides a global point of access to it.	Logging, configuration, or resource manager.
Factory Method	Lets subclasses decide which class to instantiate. Used when the exact type of object isn't known in advance.	UI components that depend on platform (e.g., Windows vs Mac button).
Abstract Factory	Provides an interface for creating **families of related objects** without specifying their concrete classes.	UI themes (e.g., light vs dark mode component sets).
Builder	Separates the construction of a complex object from its representation, so the same construction process can create different representations.	Building a complex document or a meal with optional parts.
Prototype	Creates new objects by **copying an existing object (a prototype)** rather than creating from scratch.	Cloning game characters or configuration templates.

4. Compare and contrasts the following patterns:
   (a) Abstract factory
   (b) Factory method
   (c) Adapter
   (d) Decorator

Both **Abstract Factory** and **Factory Method** are **creational patterns**, meaning they focus on object creation. However, they differ in their approach:

Feature	Abstract Factory	Factory Method
Purpose	Creates **families** of related objects	Creates a **single** object
Implementation	Uses a factory class with multiple factory methods	Uses a single factory method in a subclass
Flexibility	Provides a consistent way to create related objects	Allows subclasses to define object creation
Example	GUI toolkit that creates buttons and textboxes for different OS	A game that creates different enemy types based on difficulty

Key Difference:

- **Abstract Factory** is used when multiple related objects need to be created together.
- **Factory Method** is used when a single object needs to be created dynamically.

Both **Adapter** and **Decorator** are **structural patterns**, meaning they help organize relationships between objects. However, they serve different purposes:

Feature	Adapter	Decorator
Purpose	**Converts** one **interface** into another	**Adds behavior** dynamically to an object
Implementation	Wraps an existing object to match a required interface	Wraps an object to extend its functionality
Flexibility	Helps integrate incompatible interfaces	Allows dynamic addition of features
Example	A power adapter that lets a US plug work in a European socket	A coffee shop where customers add milk, sugar, or caramel to their coffee

Key Difference:

- **Adapter** is used for compatibility—making an object work with an expected interface.
- **Decorator** is used for extending functionality—adding features dynamically.

5. Provide a real-life example for each of the following patterns. Discuss each of them.
   (a) Singleton
   (b) Factory method
   (c) Strategy
   (d) Decorator
   (e) Chain of responsibility

Patterns	Definition	Key Characteristics	Example Use Cases
Singleton	The Singleton Pattern ensures that a class has **only one instance** and provides a global point of access to it. This is useful when exactly one object is needed to coordinate actions across a system.	**Private Constructor**: Prevents direct instantiation **Static Instance**: Holds the single instance of the class **Global Access Method:** Provides controlled access to the instance	**Data Connection Pool:** Ensures only one connection manager exists. **Logging Service:** Centralized logging across an application. **Configuration Manager:** Stores global settings
Factory Method Pattern	The Factory Method Pattern provides an interface for creating objects but **allows subclasses to decide** which class to instantiate. This promotes loose coupling and enhances flexibility.	**Encapsulates Object Creation:** Clients don't need to know the exact class **Supports Polymorphism:** Different implementations can be returned dynamically **Improves Maintainability:** New object types can be added without modifying existing code.	**Document Creation in Microsoft Office:** Word, Excel and PowerPoint documents are created dynamically **Game Development:** Different enemy types are instantiated based on difficulty level **Data Parsers:** JSON, XML and CSV parsers are created based on file type
Strategy Pattern	The Strategy defines a **family of algorithms**, encapsulates them, and makes them interchangeable. This allows dynamic selection of selection of algorithms at runtime	**Encapsulates Algorithms:** Each strategy is a separate class **Promotes Flexibility:** Algorithms can be swapped without modifying client code **Supports Open-Closed Principle:** New strategies can be added without altering existing logic	**Payment Processing in E-Commerce:** Credit Card, PayPal and cryptocurrency payments are handled dynamically **Sorting Algorithms:** Different sorting methods (QuickSoft, MergeSort) are chosen based on data characteristics **AI Decision-Making:** Different strategies are applied based on game conditions

Patterns	Definition	Key Characteristics	Example Use Cases
Decorator Pattern	The Decorator Pattern allows behaviour to be **added dynamically** to objects without modifying their structure. It provides a flexible alternative to subclassing.	**Wraps Objects:** Each decorator extends functionality without altering the base object. **Supports Composition:** Multiple decorators can be stacked. **Enhances Modularity:** New features can be added without modifying existing code.	**Coffee Customization in Starbucks:** Customers start with a basic coffee and add milk, sugar, or caramel dynamically **UI Components:** Scrollbars, borders, and shadows are added to graphical elements **File I/O Streams:** Compression and encryption are applied to file streams
Chain of Responsibility Pattern	The Chain of Responsibility Pattern allows multiple objects to **handle a request sequentially**. If one object cannot process the request, it passes it to the next handler	**Decouples Senders and Receivers:** Handlers process requests independently **Supports Dynamic Processing:** Requests are passed along a chain until handled **Enhances Flexibility:** New handlers can be added without modifying existing logic.	**Customer Support System:** Requests are handled by chatbot → junior agent → senior agent →manager **Logging Frameworks:** Different log levels (INFO, WARNING, ERROR) are processed sequentially. **Middleware Processing:** HTTP requests pass through authentication, validation and logging layers.

6. Compare design pattern with software framework.

Aspect	Design Pattern	Software Framework
Definition	A **general, reusable solution** to a commonly occurring problem in software design. It's a concept, not code.	A **concrete implementation** that provides a reusable set of libraries or components to build applications.
Nature	Conceptual/architectural blueprint	Executable code/libraries
Usage	Applied manually by the developer based on need.	Used by plugging your code into its structure (often inversion of control).
Flexibility	Highly flexible and adaptable.	Less flexible due to predefined structure.
Examples	Singleton, Observer, Factory, Façade, Strategy (as a pattern)	Angular, Spring, Django, .NET, React (as a framework).
Scope	Solves **specific design problems** in small or large parts of a system.	Provides **complete structure** for application development.
Learning Curve	Generally lighter – involves understanding concepts.	Steeper – requires learning APIs, tools, conventions.
Control Flow	You are in control – you call the pattern logic.	Framework is in control – it calls your code (inversion of Control).

Summary:

- **Design Pattern** = *How* to solve a problem in a reusable way (idea, practice).
- **Framework** = *Where* to put your code and *what* tools to use to build software (infrastructure).

Additional Questions (Home Work)

a) An ice cream shop is selling various type of ice cream. Customer may buy for the basic type where there is no topping on the ice cream. Customer may request to add different types of topping on the ice-cream. There are three types of topping, chocolate, peanut, raisin. Assume that the basic ice cream costs $1.50 each scoop, and each topping has different cost. Chocolate topping is $0.60, peanut topping is $0.50 and raisin topping is $0.45. A program is created to calculate the total costs for each purchase of the ice-cream.

(i) Suggest **ONE (1)** design pattern which is suitable for this scenario. Justify your suggestion.

(1 + 4 marks)

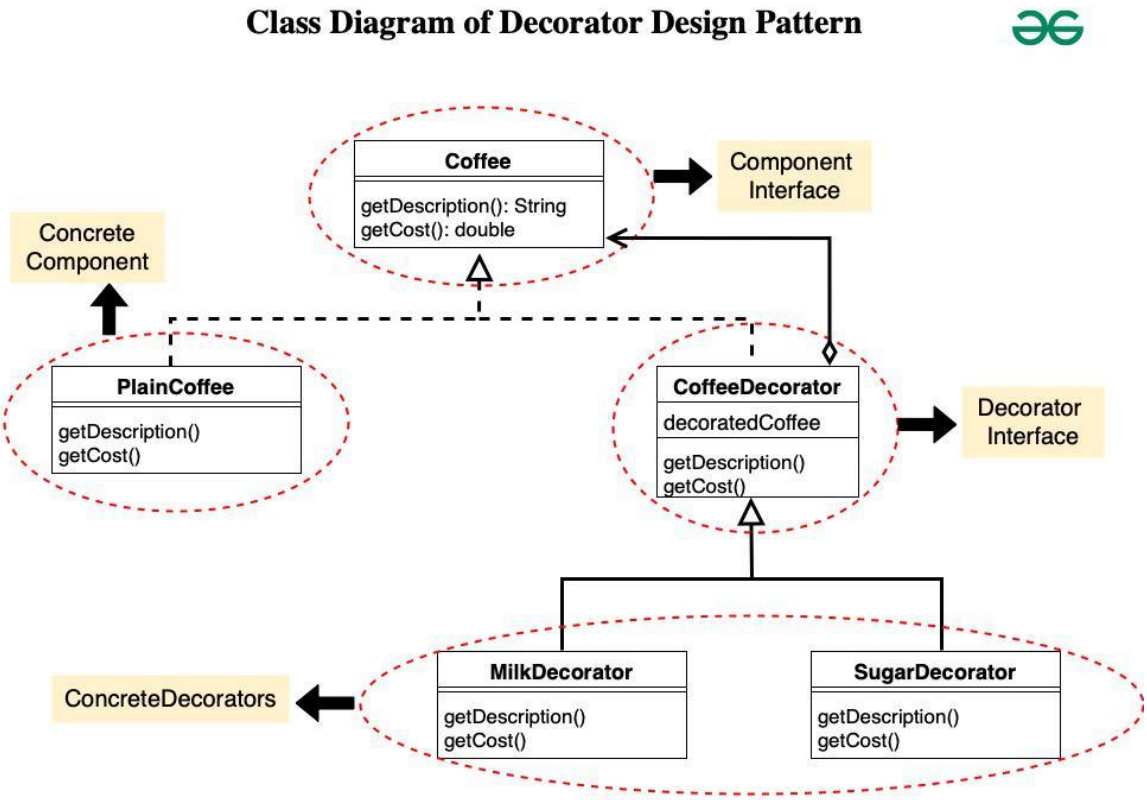
Suggested Design Pattern: Decorator Pattern

Justification:

The **Decorator Pattern** is ideal for this ice cream scenario because:

- It allows you to add **toppings dynamically** to the base ice cream object.
- Each topping (chocolate, peanut, raisin) **adds additional behavior** (cost) to the original object **without modifying the base class**.
- You can **combine multiple toppings flexibly**, even at runtime.
- It adheres to the **Open/Closed Principle** — open for extension, but closed for modification.

(ii) Draw a UML class diagram to illustrate the design pattern for the program structure.


Class Diagram of Decorator Design Pattern
Coffee
getDescription(): String
getCost(): double
Component Interface
Concrete Component
PlainCoffee
getDescription()
getCost()
CoffeeDecorator
decoratedCoffee
getDescription()
getCost()
Decorator Interface
MilkDecorator
getDescription()
getCost()
SugarDecorator
getDescription()
getCost()
ConcreteDecorators

b) A newly established development company is appointed to develop a vaccination scheduling software. The software is used to process the vaccination request of a person. When the person registers and request for vaccination, the person is required to input his information such as name, identity card number, personal health information such as medication, surgical history, and so on. In addition, there are different types of requests according to the age of the person, country of origin and nature of work. The request not only consists of personal information, it also consists of the information for/from each sub-process of vaccination. The type of request and its information will be sent to a series of processes but not all processes will be used to handle a request type. The process will pass the request to other processes if it finds that the request is not intended for itself. In each process, the information will be processed by a series of functions related to the information provided. The company has employed a software architect as a contract staff for the project. The software architect is in the midst of deciding on whether to apply the *chain of responsibility* or *mediator* design patterns.

(i)

Chain of responsibility Chain of responsibility allows a request to be sent to group of objects and allow the related object(s) to decide which should process the request.

(ii)

Abstract factory.

Builder.